

SNORT IDS SETUP & ATTACK DETECTION

1. Network Infrastructure & Environment Topology

The simulation environment was developed entirely within isolated virtual switching fabrics on VMware Workstation, placing all systems within the same subnet to mimic an internal corporate segment or localized industrial Demilitarized Zone (DMZ).

Host Machine Role	Operating System	IP Address Assignment	Primary Interface Name	Functional Scope in Lab
Intrusion Detection System (IDS)	Ubuntu	192.168.127.127	ens33	Sniffs traffic in Promiscuous Mode and matches packets against local.rules to generate real-time alerts
Adversary Node (Attacker)	Kali Linux 2025.4	192.168.127.128	eth0	Launches Nmap scans, hping3 SYN floods, Telnet exploits, and SQL Injection payloads
Vulnerable Endpoint (Target)	Metasploitable 2 Linux	192.168.127.129	eth0	Hosts vulnerable services (Telnet on port 23, HTTP on port 80) to receive attacks and verify IDS detection

2. Snort Rule Engineering (rules/local.rules)

To precisely log individual adversarial actions without letting automated fast scans bleed into generic volumetric alerts, a set of fine-tuned signatures was declared inside `/etc/snort/rules/local.rules`. These rules monitor specific Layer 4 flags, rate limits, and application-layer pattern matches.

```
Jun 15 09:53
ubuntu@ubuntu-virtual-machine: ~
GNU nano 6.2 /etc/snort/rules/local.rules *
# 1. Nmap Scan
alert tcp any any -> any any (msg:"Nmap TCP Scan Detected"; flags:S; threshold:type threshold, track by_src, count 2, seconds 10; sid:1000001; rev:1;)

# 2. Telnet Access
alert tcp any any -> any 23 (msg:"Telnet Access Attempt"; flags:S; sid:1000002; rev:1;)

# 3. DoS Attack
alert tcp any any -> any any (msg:"Potential SYN Flood DoS Attack"; flags:S; threshold:type threshold, track by_dst, count 20, seconds 5; sid:1000003; rev:1;)

# 4. SQL Injection
alert tcp any any -> any 80 (msg:"Possible SQL Injection Attempt"; content:"UNION"; nocase; sid:1000004; rev:1;)
```

3. Attack Scenarios, Execution & Logs Analysis

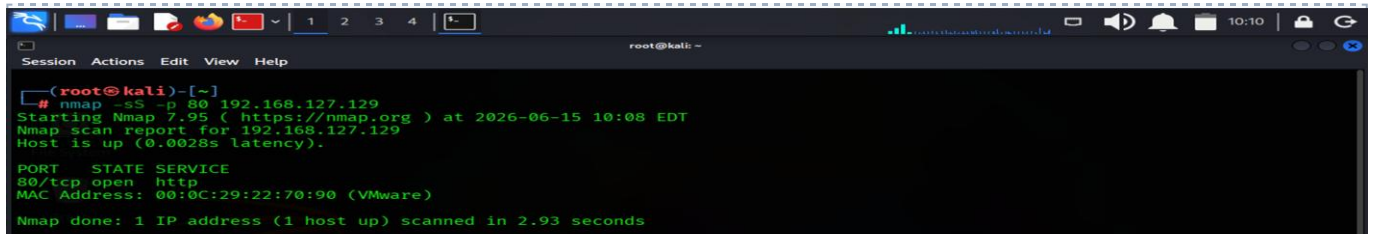
The lab required individual validation of each vector. Below is the technical breakdown of the adversarial actions executed from Kali Linux and the corresponding high-fidelity alert generation inside Snort console output.

Scenario 3.1: Nmap TCP Stealth Scan (SYN Scan)

The adversary utilized selective port checking to build an active inventory of open daemons on the target machine. By restricting scanning parameters, the attacker attempted to slip under common log mechanisms.

```
# Command Executed on Kali Linux Terminal
nmap -sS -p 80 192.168.127.129
```

Analytical Response: Snort immediately tracked incoming connection requests that left the Three-Way Handshake incomplete. Because the source address hit multiple port vectors within the 10-second window, SID 1000001.



```
(root@kali)-[~]
# nmap -sS -p 80 192.168.127.129
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-15 10:08 EDT
Nmap scan report for 192.168.127.129
Host is up (0.0028s latency).

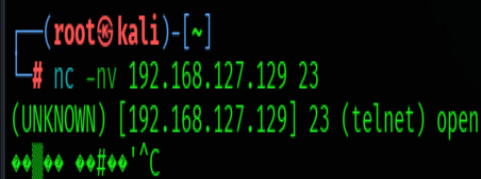
PORT      STATE SERVICE
80/tcp    open  http
MAC Address: 00:0C:29:22:70:90 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 2.93 seconds
```

Scenario 3.2: Telnet Active Session Exploitation

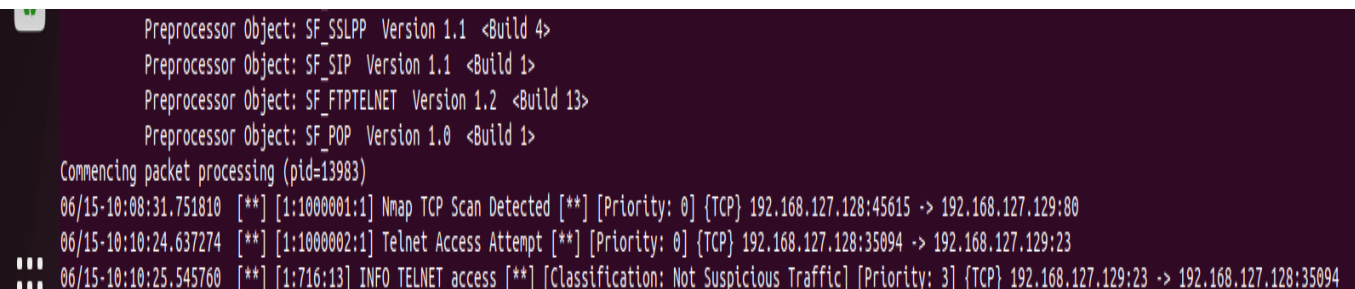
An explicit attempt was initiated to check for remote shell access or perform credential testing against the clear-text management service (Telnet) hosted on Metasploitable 2.

```
# Command Executed on Kali Linux Terminal
nc -nv 192.168.127.129 23
```



```
(root@kali)-[~]
# nc -nv 192.168.127.129 23
(UNKNOWN) [192.168.127.129] 23 (telnet) open
♦♦♦♦♦♦♦♦♦♦
```

Analytical Response: The IDS engine analyzed the TCP synchronization phase initiated toward destination port 23. Since the inbound packet pattern matched the strict rule parameter (flags:S), Snort immediately flagged the session as unauthorized access, triggering SID 1000002



```
Preprocessor Object: SF_SSLPP Version 1.1 <Build 4>
Preprocessor Object: SF_SIP Version 1.1 <Build 1>
Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>
Preprocessor Object: SF_POP Version 1.0 <Build 1>
Commencing packet processing (pid=13983)
06/15-10:08:31.751810  [**] [1:1000001:1] Nmap TCP Scan Detected [**] [Priority: 0] {TCP} 192.168.127.128:45615 -> 192.168.127.129:80
06/15-10:10:24.637274  [**] [1:1000002:1] Telnet Access Attempt [**] [Priority: 0] {TCP} 192.168.127.128:35094 -> 192.168.127.129:23
06/15-10:10:25.545760  [**] [1:716:13] INFO TELNET access [**] [Classification: Not Suspicious Traffic] [Priority: 3] {TCP} 192.168.127.129:23 -> 192.168.127.128:35094
```

Scenario 3.3: Volumetric SYN Flood Denial of Service (DoS)

To verify the detection capability of the Snort IDS engine, a high-rate SYN Flood attack was executed using hping3. The attacker flooded port 80 with continuous SYN packets to trigger the corresponding alert

```
# Command Executed on Kali Linux Terminal
hping3 -S -p 80 --flood 192.168.127.129
```

```
(root@kali)-[~]
# hping3 -S -p 80 --flood 192.168.127.129
HPING 192.168.127.129 (eth0 192.168.127.129): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
^C
— 192.168.127.129 hping statistic —
9372 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

Analytical Response: The destination tracking rule (SID 1000003) immediately intervened. When the rate of incoming packets exceeded 20 packets within 5 seconds, Snort logged a severe volumetric condition alert.

```
? Preprocessor Object: SF_SSH Version 1.1 <Build 3>
Preprocessor Object: SF_SSLPP Version 1.1 <Build 4>
Preprocessor Object: SF_SIP Version 1.1 <Build 1>
Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>
Preprocessor Object: SF_POP Version 1.0 <Build 1>
Commencing packet processing (pid=13983)
06/15-10:08:31.751810 [**] [1:1000001:1] Nmap TCP Scan Detected [**] [Priority: 0] {TCP} 192.168.127.128:45615 -> 192.168.127.129:80
06/15-10:10:24.637274 [**] [1:1000002:1] Telnet Access Attempt [**] [Priority: 0] {TCP} 192.168.127.128:35094 -> 192.168.127.129:23
06/15-10:10:25.545760 [**] [1:716:13] INFO TELNET access [**] [Classification: Not Suspicious Traffic] [Priority: 3] {TCP} 192.168.127.129:23 -> 192.168.127.128:35094
06/15-10:11:54.873647 [**] [1:1000001:1] Nmap TCP Scan Detected [**] [Priority: 0] {TCP} 192.168.127.128:1942 -> 192.168.127.129:80
06/15-10:11:54.876716 [**] [1:1000001:1] Nmap TCP Scan Detected [**] [Priority: 0] {TCP} 192.168.127.128:1944 -> 192.168.127.129:80
06/15-10:11:54.878464 [**] [1:1000001:1] Nmap TCP Scan Detected [**] [Priority: 0] {TCP} 192.168.127.128:1946 -> 192.168.127.129:80
06/15-10:11:54.880627 [**] [1:1000001:1] Nmap TCP Scan Detected [**] [Priority: 0] {TCP} 192.168.127.128:1948 -> 192.168.127.129:80
06/15-10:11:54.882386 [**] [1:1000001:1] Nmap TCP Scan Detected [**] [Priority: 0] {TCP} 192.168.127.128:1950 -> 192.168.127.129:80
06/15-10:11:54.884089 [**] [1:1000001:1] Nmap TCP Scan Detected [**] [Priority: 0] {TCP} 192.168.127.128:1952 -> 192.168.127.129:80
06/15-10:11:54.885780 [**] [1:1000001:1] Nmap TCP Scan Detected [**] [Priority: 0] {TCP} 192.168.127.128:1954 -> 192.168.127.129:80
06/15-10:11:54.886919 [**] [1:1000001:1] Nmap TCP Scan Detected [**] [Priority: 0] {TCP} 192.168.127.128:1956 -> 192.168.127.129:80
06/15-10:11:54.888664 [**] [1:1000001:1] Nmap TCP Scan Detected [**] [Priority: 0] {TCP} 192.168.127.128:1958 -> 192.168.127.129:80
```

Scenario 3.4: Web Application SQL Injection Vulnerability Inspection

An application-layer exploitation attempt was simulated targeting a database query engine by injecting structured ANSI SQL strings into HTTP request parameters.

```
# Command Executed on Kali Linux Terminal
curl "http://192.168.127.129/?id=UNION+SELECT"
```

```
root@kali: ~
Session Actions Edit View Help
(root@kali)-[~]
# curl "http://192.168.127.129/?id=UNION+SELECT"
<html><head><title>Metasploitable2 - Linux</title></head><body>
<pre>
```

Analytical Response: Snort inspected the data payload area of the TCP segment. It found the text string "UNION", which triggered the non-case-sensitive signature match inside SID 1000004.

```
06/15-10:11:56.566410 [**] [1:1000001:1] Nmap TCP Scan Detected [**] [Priority: 0] {TCP} 192.168.127.128:11310 -> 192.168.127.129:80
06/15-10:11:56.566866 [**] [1:1000001:1] Nmap TCP Scan Detected [**] [Priority: 0] {TCP} 192.168.127.128:11312 -> 192.168.127.129:80
06/15-10:17:33.543948 [**] [1:1000004:1] Possible SQL Injection Attempt [**] [Priority: 0] {TCP} 192.168.127.128:58700 -> 192.168.127.129:80
```

4. Conclusion and Engineering Takeaways

The implementation successfully validates the capability of an open-source Network IDS to detect diverse network threats. By setting clear boundaries for thresholds and looking deep into packet content, the platform tracked activities accurately from network reconnaissance up to layer 7 web application exploits.

In an enterprise defense network, these console alerts should feed into a central Security Information and Event Management (SIEM) system. This setup helps security analysts quickly orchestrate automated firewall blocks or block malicious sources automatically via an Intrusion Prevention System (IPS).